Guion de presentación

Microhubs y comercio de proximidad · Equipo 04
31 diapositivas · Arquitectura actual de 10 microservicios · 13 octubre 2026

01. Microhubs y comercio de proximidad

Presentamos nuestra plataforma de microhubs y comercio de proximidad. La idea es acercar los productos al cliente mediante puntos de distribución en su zona. En esta presentación explicaremos la arquitectura actual, que ya cuenta con diez microservicios, y después mostraremos su funcionamiento en vivo.

02. Problema y objetivo

Las tres columnas resumen el problema, la solución y el objetivo del parcial. Buscamos reducir las dificultades de atender pedidos pequeños y controlar la disponibilidad por zona. Para hacerlo, integramos el portal con diez servicios que comparten seguridad y almacenamiento persistente.

03. Alcance y requerimientos del parcial

En esta diapositiva separamos los requisitos funcionales de los técnicos. El sistema cubre pedidos, entregas, planeación, documentos y avisos. La arquitectura utiliza diez servicios, tres motores de datos y Docker. Las APIs también atienden Android con JSON y escritorio con XML; TLS y el despliegue en nube quedan para la siguiente etapa.

04. Correcciones del primer parcial

Los cuatro apartados muestran las mejoras del proyecto. Ampliamos la planeación y los flujos de operación, conectamos diez servicios por HTTP y actualizamos la documentación. Las evidencias adjuntas registran 93 casos, que sirven para comprobar respuestas y cambios en los datos.

05. Evolución del sistema web

Las tres columnas muestran cómo creció la web. Operación reúne pedidos, inventario y validaciones; Administración incorpora reportes, archivos y avisos; Planeación permite evaluar ubicaciones, costos y capacidad. Estas funciones del portal conviven con las APIs de los microservicios.

06. Perfiles, permisos y ámbito de datos

La lista muestra que cada perfil tiene un alcance distinto. El cliente consulta sus pedidos, el operador trabaja con su microhub y el repartidor con sus entregas. Los perfiles de planeación, auditoría y análisis tienen funciones específicas. El administrador gestiona usuarios, permisos y el estado de los servicios.

07. Arquitectura implementada

En el diagrama, el navegador entra al portal Flask por el puerto 5000. La web y los clientes HTTP se conectan a los diez microservicios, del 5101 al 5110, usando HTTP y JWT. Abajo están PostgreSQL, MongoDB y Redis. Compartimos estos almacenes según la operación, y el portal conserva funciones que trabajan directamente con ellos.

08. Responsabilidades de los diez servicios

La tabla resume la distribución actual. Catálogos se separó en Productos, Zonas y Microhubs, para que cada servicio tenga una responsabilidad concreta. Seguimiento ahora se encarga del recorrido GPS. Auth, Usuarios, Pedidos, Entregas, Planeación y Documentos completan los diez servicios. Los avisos siguen en la web y PostgreSQL, sin un microservicio de Notificaciones.

09. Diseño interno y componentes comunes

Los cuatro bloques explican la estructura del código. Cada servicio define sus rutas y reglas en su app.py. Los módulos comunes concentran seguridad, validación y formatos. El acceso a datos usa el cliente correspondiente a cada motor, y los contratos y trazas ayudan a documentar y revisar las peticiones.

10. Un Dockerfile por microservicio

Cada microservicio tiene su propio Dockerfile. Como muestran los apartados, parte de Python 3.12, instala dependencias e incluye el código y los contratos. Gunicorn ejecuta dos workers con un usuario sin privilegios. EXPOSE identifica el puerto y el healthcheck revisa que el servicio responda.

11. Diez contenedores y puertos propios

Esta tabla muestra el contenedor y puerto de cada servicio. Los cambios clave son Productos en 5103, Zonas en 5108, Microhubs en 5109 y Seguimiento en 5110. El puerto 5108 ya no corresponde a Notificaciones. Esta separación permite ejecutar y revisar cada responsabilidad en su propio contenedor.

12. Despliegue, redes y persistencia

El diagrama muestra la web y los diez servicios conectados a la red interna de datos. PostgreSQL y MongoDB conservan información en volúmenes; Redis está configurado sin persistencia. Las bases no publican puertos al equipo. Compose espera a que PostgreSQL esté saludable y a que terminen las migraciones antes de iniciar los servicios dependientes.

13. Configuración y operación con Compose

Los primeros dos apartados muestran la configuración: una llave común para JWT, conexiones a las bases y una plantilla para localizar cada servicio por nombre y puerto. El comando de inicio construye y levanta el entorno en segundo plano. El comando down detiene los contenedores y conserva los volúmenes al no incluir la opción -v.

14. Un pedido de principio a fin

El flujo empieza con el pedido del cliente y continúa con la preparación del operador. Entregas registra el cierre, el cobro y las referencias de evidencia. PostgreSQL actualiza el estado y genera el aviso mediante un trigger. Seguimiento registra el GPS y Documentos guarda los archivos; los identificadores permiten relacionar todo con el mismo pedido.

15. Secuencia del cierre y comunicación

En la secuencia, el cliente envía el resultado a Entregas. El servicio valida la sesión y la revocación en Redis, y registra el cierre en una transacción de PostgreSQL, junto con el historial y el aviso generado por el trigger. El primer cierre devuelve 201; repetir el mismo UUID devuelve 200 sin aplicar el cierre dos veces. No existe una transacción global entre PostgreSQL y MongoDB.

16. Cada almacén tiene una función

Las tres columnas muestran la función de cada almacén. PostgreSQL conserva los datos relacionados del negocio y sus transacciones. MongoDB almacena documentos, GPS, simulaciones y registros. Redis guarda sesiones, revocaciones, caché y contadores temporales. Elegimos cada motor según las características de los datos.

17. Modelo lógico y físico de PostgreSQL

En el modelo se relacionan el usuario, el pedido, sus detalles, los productos y la entrega. El inventario se identifica por microhub y producto. Las claves mantienen las relaciones, mientras que restricciones, triggers y funciones aplican reglas. Los bloqueos protegen existencias y el UUID de operación ayuda a evitar cierres duplicados.

18. MongoDB: operaciones del proyecto

Los cuatro apartados muestran operaciones reales de MongoDB. Documentos inserta y recupera archivos, y Seguimiento guarda puntos GPS en ubicaciones_entrega. Las sincronizaciones se actualizan por UUID. Las agregaciones permiten resumir recorridos y simulaciones, mientras que los índices y validadores organizan y restringen los datos.

19. Redis: claves, TTL y efecto

Cada fila muestra una clave de Redis y su efecto. La sesión tiene un TTL inicial de 900 segundos; la revocación bloquea un JWT durante su vigencia restante. El refresh dura hasta siete días y se usa una sola vez. La caché de catálogo dura cinco minutos y los contadores de consumo permiten devolver 429 al superar el límite.

20. JWT, permisos y sesión común

La tabla resume las capas de validación. Primero comprobamos la firma y vigencia del JWT. Después Redis confirma que la sesión siga activa y que el token no esté revocado. Finalmente revisamos permisos y alcance de los datos. Por eso una sesión válida no significa que el usuario pueda consultar cualquier pedido o entrega.

21. Contratos JSON y XML

Las dos columnas muestran que una misma consulta de productos puede responder en JSON o XML según Accept. Content-Type indica el formato del cuerpo enviado. El XML de entrada se valida con XSD. Hay una excepción relevante: el envío de lotes de puntos GPS en Seguimiento admite únicamente JSON.

22. Detalle de los contratos de integración

Estos apartados detallan el contrato. JSON define tipos, campos y errores; las listas incluyen datos y paginación. XML conserva raíz, versión y namespace. Los nueve XSD validan las entradas XML, y se rechazan DTD y entidades. El número de esquemas no tiene que coincidir con los diez servicios, porque algunos comparten contratos.

23. Swagger y salud de los diez servicios

Cada servicio, desde el puerto 5101 hasta el 5110, tiene su propia página en /api/v1/docs. Swagger permite cargar el JWT con Authorize y probar las rutas. OpenAPI describe los parámetros y respuestas. Para revisar disponibilidad usamos /health o /api/v1/salud, que muestran el servicio y sus dependencias.

24. Evidencia completa por endpoint

Los cinco apartados indican qué guardamos como evidencia. Incluimos la solicitud, la respuesta completa, el tiempo y la traza. También comprobamos qué quedó en las bases de datos. Los casos cubren operaciones correctas y errores, y se conservan en archivos HTML, Markdown y JSON para poder revisarlos.

25. Evidencia real: cierre de una entrega

Este ejemplo corresponde a la entrega 129 y al pedido 153. A la izquierda se ve el cobro de 146.80 y el UUID de la operación. A la derecha, el primer cierre responde 201 y marca duplicado como falso. Repetir el UUID devuelve 200 y duplicado verdadero; un monto incorrecto devuelve 422. Son resultados de la evidencia adjunta.

26. Errores, validaciones y observabilidad

La diapositiva agrupa los errores por causa: datos inválidos, problemas de acceso, conflictos, límites y dependencias no disponibles. El cuerpo de error mantiene una estructura común. X-Traza-Id conecta la respuesta HTTP con los registros y métricas, lo que facilita localizar una petición al investigar una falla.

27. Integración desde el portal web

Esta tabla representa la integración del código actual. El tablero consulta la salud de los diez servicios, los resúmenes y un ejemplo XML desde Zonas. La vista del pedido combina Pedidos, Entregas, Seguimiento y Documentos. Cada llamada utiliza el JWT del usuario y registra código HTTP, tiempo y traza. Si un servicio falla, el portal informa el problema y conserva el resto de su funcionamiento.

28. Pruebas unitarias y de integración

Los cuatro bloques distinguen las pruebas: catorce unitarias, siete de contrato y cinco de integración web, además de los 93 casos de evidencia. Las de contrato recorren los diez servicios y verifican perfiles y formatos. Las pruebas automáticas usan MongoDB simulado, así que sus validadores requieren comprobación adicional con el motor real.

29. Pruebas y resultados registrados

El resultado guardado en el proyecto es de 93 casos aprobados de 93. Incluye respuestas correctas, errores controlados y comprobaciones en los tres almacenes. Este resultado pertenece a la ejecución registrada del 13 de octubre; la demostración en vivo permitirá comprobar el entorno que estamos utilizando ahora.

30. Trazabilidad: requisito, componente y prueba

La tabla relaciona cada requisito con el componente y la evidencia que lo respalda. Por ejemplo, el despliegue se verifica con los diez Dockerfiles y la salud de los contenedores. Los contratos se comprueban con casos JSON y XML, y el cierre de entregas con casos de validación, éxito, repetición y conflicto.

31. Resultados, problemas y pendientes

Para cerrar, la arquitectura actual separa los catálogos, mueve el GPS a Seguimiento y conserva los avisos mediante PostgreSQL y la web. El proyecto registra diez servicios y 93 casos aprobados. Quedan aspectos por comprobar o mejorar, como la caché y los validadores de MongoDB real. La siguiente etapa contempla TLS, secretos propios, nube y pruebas de carga.